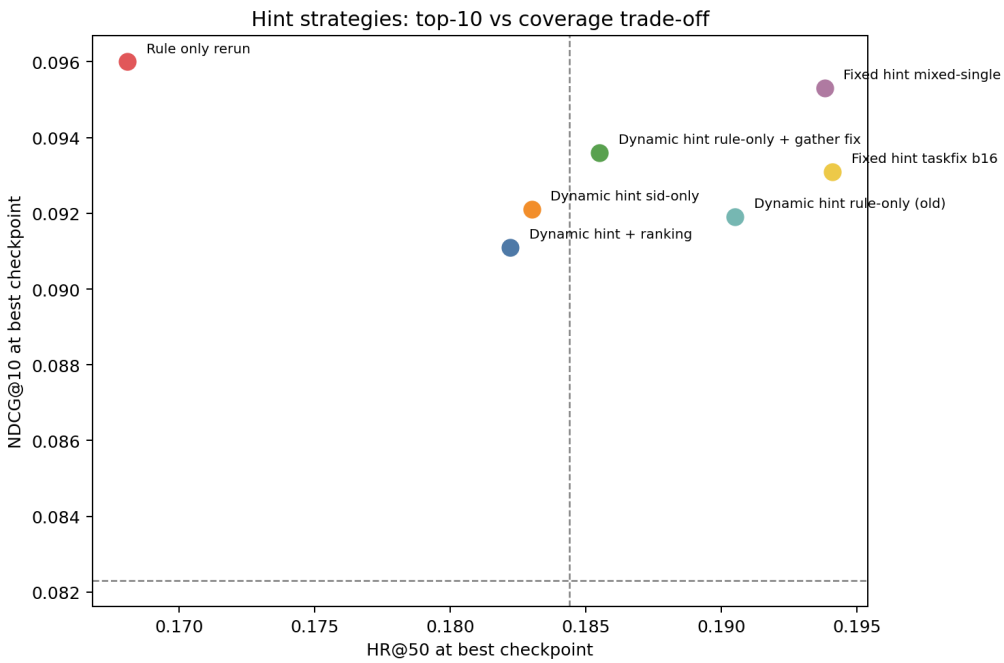
深度研究报告

GenRec Instruments-grec RL 实验深度复盘

Reward Form、Hint Scaffold 与 Offline/Online Gap 的代码对照分析

Codex

2026-03-28



核心问题：在当前 Instruments-grec 结果上，哪些 RL 设计真正有效，当前最可信的 contribution 是什么，以及为什么离线 hint 结果显著好于在线最终评测？

研究范围：覆盖 20 个用户指定结果目录、SFT 基线、离线 beam16 hint bundle、以及当前仓库相关训练与评测代码；证据截止到 2026-03-28 本地仓库快照。

目标读者：GenRec 内部研究与实验推进者，目标是决定 paper framing、主打实验和下一轮 ablation。

证据窗口：本地结果与代码访问日期：2026-03-28

目录

1 执行摘要 **3**

2 研究范围与方法 **3**

3 实验版图与目录口径 **4**

3.1 版本时间线与 bug 对齐 5

4 主结果一：Reward Form Ablation **5**

4.1 rule-only 仍然是当前最强的无 hint top-10 基线 6

4.2 纯 prefix reward 不能替代 exact reward 6

4.3 token-level prefix advantage 不是不能用，但必须处理归一化 7

4.4 errtok 版本目前没有带来额外收益 7

4.5 reward 线的可信结论 7

5 主结果二：Hint Scaffold Ablation **8**

5.1 fixed hint 是当前最平衡、也最值得写成主结果的方向 9

5.2 task-aware fixed hint map 更安全，且仍然保住了 coverage 9

5.3 旧 fixed 错版为什么反而更好：它本质上是一个更浅的 hint curriculum 10

5.4 dynamic hint 有价值，但 canonical 版本应以 gather-fix 为准 12

5.5 sid-only 更直接对齐评测任务，但目前仍未优于 mixed-task 13

5.6 ranking 没有显示出明确收益，而且有机制层面的解释 14

6 为什么 fixed 当前比 dynamic 更好 **15**

6.1 先把误解去掉：dynamic 的“变化”不是错，问题在于 gate 与优化目标不一致 15

6.2 fixed 比 dynamic 强，不是因为 fixed 理论更对，而是因为当前 dynamic gate 太脆 . . 16

6.3 为什么错版 fixed 反而更好：少一些 hint 可能更接近最终目标，但前提是要稳定 . . . 17

6.4 如果要让 dynamic 真正赢 fixed，它需要改 stopping signal，而不是只靠在线变化 . . . 17

7 代码层解释：为什么 ranking 不明显、为什么 prefix shaping 容易不稳定 **17**

7.1 ranking reward 对 hardest case 几乎没有帮助 17

7.2 prefix shaping 的问题不在“粒度”，而在“credit assignment” 18

8 我们现在最可信的 contribution 是什么 **18**

9 后续应该怎么做 **19**

9.1 优先级最高：把 fixed hint 变成真正的主线方法 19

9.2 第二优先级：把 dynamic hint 变成 scheduler ablation，而不是主结果 19

9.3 第三优先级：reward 线只保留最必要的四条 20

9.4 任务设计层面：不要过早只押 sid-only，要先做 clean ablation 20

10 结论与后续问题 **20**

10.1 已经被强支持的结论 20

10.2 较大概率为真，但还需要补 cleaner ablation 的结论 21

10.3 仍然未知、值得马上验证的问题 21

A 用户指定目录的分类索引 **21**

B Legacy / Superseded Run 结果表 **22**

C 来源清单 **22**

1 执行摘要

一句话结论

> 当前最可信、也最值得写成主线 *contribution* 的结论不是“复杂 *reward shaping*”，而是“***task-aware oracle scaffold* 能比复杂 *reward* 形状更稳定地缓解 *semantic-ID RL* 的稀疏性**”。在现有结果里，***fixed hint + rule-only*** 是最平衡的方向：它几乎追平最强 *top-10* 指标，同时显著修复了 *plain rule-only* 的 *top-50 coverage* 损失。

- 无 hint 的 reward form 对比里，best `NDCG@10` 仍然来自 **rule-only rerun**：`0.0960`；它比 baseline mixed GRPO 高 `+0.0008`，比 `prefix only` 高 `+0.0098`，说明 exact reward 在当前设定下并没有像直觉上那样拖垮 top-10。
- 但 plain rule-only 的代价是 coverage 收缩：best `HR@50=0.1681`，低于 SFT 基线的 `0.1844`；这个结构性问题在 mixed GRPO 上也存在。
- **fixed hint mixed-single** 的 best 点达到 `NDCG@10=0.0953`、`HR@10=0.1193`、`NDCG@50=0.1114`、`HR@50=0.1938`。相对 plain rule-only，它只损失 `0.0007` 的 `NDCG@10`，却换来 `+0.0257` 的 `HR@50` 增益。
- reward shaping 不是完全没用，但高度依赖 credit assignment 细节。最明显的证据是 `prefix tokenadv raw` 几乎崩掉，而加上 `totalnorm` 后 `NDCG@10` 直接回升 `+0.0430`。
- fixed 当前赢 dynamic，核心更像是“**稳定但略偏的 scaffold**”打赢了“**自适应但由 one-hit-any 门控驱动的抖动 scaffold**”，而不是 dynamic 这个方向在理论上更差。
- “ranking 好像不加比较好”这件事，当前证据支持“**至少没有显示出明确收益**”。在 dynamic hint 组里，`ranking` 版本四个主指标都弱于 `rule-only + gather fix`。代码层面也能解释：ranking reward 只有在 group 内已经出现 exact hit 时才会激活，它对最难的“全组 0 hit”样本几乎没有帮助。

2 研究范围与方法

证据清单与口径

本报告使用四类主证据：

- **S1**: 用户指定的 20 个结果目录及其 `checkpoint-*/metrics.json`，另补 SFT 基线 `results/Instruments-grec-sft-qwen4B-4-256-dsz0/checkpoint-495/metrics.json`。
- **S2–S4**: 本地 beam16 hint bundle，包括 `instruments_grec_beam16_hint_difficulty_table.csv`、cascade summary 和 task-aware fixed hint map。
- **S5–S9**: 当前仓库代码，重点是 `rewards/ranking_reward.py`、`token_prefix_grpo_trainer.py`、`trl_trainer.py`、`fixed_hint_grpo_trainer.py` 与 `evaluate.py`。
- **S10–S12**: 仓库内先前分析文档，仅作为辅助语境，不作为唯一证据来源。

本文所有“best checkpoint”默认按 `NDCG@10` 选点。相对 SFT 的相对增益定义为：

$$\Delta_{\mathrm{rel}}(m) = \frac{m_{\mathrm{variant}} - m_{\mathrm{SFT}}}{m_{\mathrm{SFT}}}$$

- m_{variant}：某个 RL 变体在其 best checkpoint 上的指标
- m_{SFT}：SFT 基线 `checkpoint-495` 上的同名指标
- $\Delta_{\mathrm{rel}}(m)$：相对提升或相对下降比例

本报告还对用户点名的全部目录做了分类：

- **主结论 run**: 当前可稳定解释、脚本与结果目录对应明确的 reward/hint ablation。
- **superseded/legacy run**: 旧 run、明显异常 run、仅单 checkpoint backup，保留作排错和演化证据，不拿来做主结论。
- **代码证据**: 用于解释 reward 定义、hint 何时参与训练/评测、以及 dynamic/fixed hint 的真正行为。

3 实验版图与目录口径

用户给的目录并不是一个完全同质的 ablation matrix，而是混合了三类东西：

- 同一想法的 **旧名/旧 run**，例如 `prefix` old alias 与后续 `prefixonly-ndcg-rule0`。
- **修复后 rerun**，例如 `rule-only-rerun-quietlog`、`prefix-seq-only-fixbool-rerun`。
- **真正新的方法线**，例如 dynamic hint、fixed hint、task-aware fixed hint、sid-only dynamic hint。

因此，合理的阅读方式不是把 20 个目录一视同仁，而是先明确主比较轴：

- **Reward form 主轴**: baseline mixed GRPO、rule-only、prefix only、prefix seq only、prefix token only、tokenadv raw、tokenadv totalnorm、tokenadv errtok。
- **Hint 主轴**: plain rule-only、dynamic hint rule-only、dynamic hint + ranking、dynamic hint sid-only、fixed hint mixed-single、fixed hint taskfix。
- **历史/排错证据**: old rule-only、old prefix-seq-only、prefix backup、single-checkpoint backup、早期 task-index fix run。

哪些 run 不该当主结论

> `Instruments-grec-grpo-rule-only-qwen...` 和 `Instruments-grec-grpo-prefix-seq-only-qwen...` 与对应 rerun 的差距极大，说明旧 run 至少混入了实现或配置问题；它们更适合证明“修复有用”，不适合证明“方法本身差”。相反，`prefix` old alias 与正式 `prefixonly-ndcg-rule0` 几乎重合，更像命名演化而不是方法差异。

3.1 版本时间线与 bug 对齐

把结果目录和脚本历史对上以后，这批实验其实分成三个阶段：

- **2026-03-17 之前后**: 旧版 dynamic 与旧版 fixed。对应目录分别是 `...dynamic-hint-cascade...` 和 `...fixed-hint-mixed-single-generate...`。这时 fixed hint map 仍按 `extra_info.index` 导出，dynamic trainer 也还没有后来的 gather 修复。
- **2026-03-20**: commit "Fix hint training map export and launch defaults" 同时修了两件事。第一，fixed hint export 从 `index` 升级为 `task+index`；第二，dynamic trainer 加入了跨 rank unresolved-group 同步与 pure rule-only 的 selected-rule-reward gather fast path。修复后默认目录变成 `...fixedhint-taskfix-b16-sft495` 和 `...dynamic-hint-cascade-reward-gather-fix...`。
- **2026-03-28**: 新增 `ranking dynamic hint` 与 `sid-only dynamic hint` 脚本。这一点很重要，因为它说明 ranking dynamic 不是跑在"旧 dynamic bug 版"上的，而是跑在 3 月 20 日修复之后的 dynamic trainer 上。

因此，后文里涉及 dynamic 的主结论都应该优先使用 `reward-gather-fix` 版本；涉及 fixed 的主结论则必须明确区分"旧 index-only hint map"与"修复后的 task+index hint map"。

4 主结果一：Reward Form Ablation

SFT 基线为：`NDCG@10=0.0823`、`HR@10=0.1094`、`NDCG@50=0.0985`、`HR@50=0.1844`。

Variant	Best ckpt	NDCG@10	HR@10	NDCG@50	HR@50
Baseline mixed GRPO	checkpoint-1665	0.0952	0.1145	0.1071	0.1696
Rule only rerun	checkpoint-2997	0.0960	0.1179	0.1070	0.1681
Prefix only	checkpoint-1332	0.0862	0.1049	0.0982	0.1612
Prefix seq only rerun	checkpoint-1332	0.0857	0.1046	0.0985	0.1636
Prefix token only	checkpoint-1665	0.0815	0.1005	0.0958	0.1668
Prefix tokenadv raw	checkpoint-999	0.0476	0.0597	0.0564	0.0991
Prefix tokenadv totalnorm	checkpoint-3326	0.0906	0.1125	0.1054	0.1809
Prefix tokenadv errtok	checkpoint-999	0.0796	0.0968	0.0914	0.1513

表 1: Reward form 主结果表。来源：S1，按 best `NDCG@10` 选点。

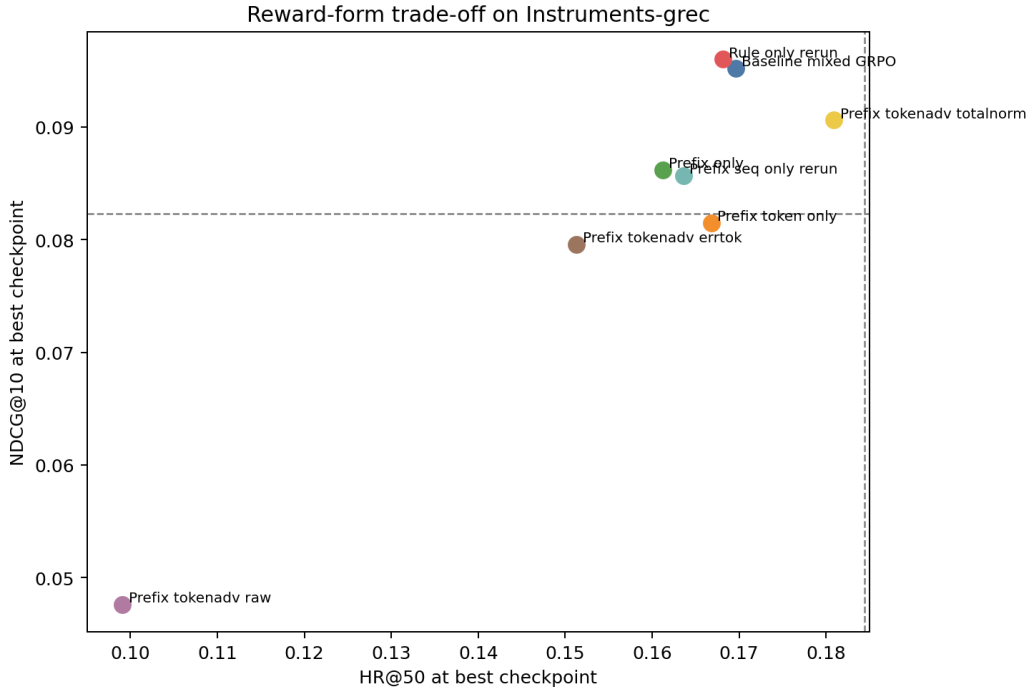


图 1: Reward 形式在 top-10 与 top-50 coverage 上的 trade-off。

4.1 rule-only 仍然是当前最强的无 hint top-10 基线

最强的无 hint top-10 指标来自 **rule-only rerun** 而不是更复杂的 shaping：

- `rule-only rerun`: `NDCG@10=0.0960`, `HR@10=0.1179`
- baseline mixed GRPO: `NDCG@10=0.0952`, `HR@10=0.1145`

二者差异不大，但方向很清楚：相较 baseline mixed GRPO，rule-only 在 `NDCG@10` 上仍有 `+0.0008` 的优势，在 `HR@10` 上有 `+0.0034` 的优势。这说明当前任务最核心的矛盾并不是“exact reward 太 sparse 所以一定打不过 dense reward”，而是“dense reward 是否真的对 hardest case 提供了有用的信用分配”。

4.2 纯 prefix reward 不能替代 exact reward

`prefix only` 和 `prefix seq only rerun` 的表现都明显弱于 rule-only：

- `prefix only`: `NDCG@10=0.0862`
- `prefix seq only rerun`: `NDCG@10=0.0857`
- `rule-only rerun`: `NDCG@10=0.0960`

绝对差距分别是 `0.0098` 和 `0.0103`。这不是噪声量级，说明“只鼓励前缀更长”并不足以支撑最后的 next-item 排序质量。结合离线 hint bundle 的结论，更合理的解释是：

- 前两位 SID token 确实决定了大多数样本的难度分层；

[0]来源：作者基于 `docs/deepresearch/genrec_rl_study_2026-03-28/data/experiment_best_metrics.csv` 自绘；底层数据来自 `results/*/checkpoint-*/metrics.json`。

- 但训练目标如果只奖励 prefix，而不继续对最终 exact item 负责，就会把“会走到正确 bucket”和“能完成最终辨识”混在一起。

4.3 token-level prefix advantage 不是不能用，但必须处理归一化

最强的工程信号来自 `tokenadv raw` 与 `tokenadv totalnorm` 的对比：

- `prefix tokenadv raw`: `NDCG@10=0.0476`, `HR@50=0.0991`
- `prefix tokenadv totalnorm`: `NDCG@10=0.0906`, `HR@50=0.1809`

也就是说，只改一层 total-token normalization，就带来：

- `NDCG@10 +0.0430`
- `HR@10 +0.0528`
- `NDCG@50 +0.0490`
- `HR@50 +0.0818`

这不是“微调 credit assignment 会有一点提升”，而是“**不做对的 normalization 会直接把训练打坏**”。

4.4 errtok 版本目前没有带来额外收益

`prefix tokenadv totalnorm errtok` 的出发点是合理的：不要把负信号打到已经正确的 prefix token 上，只惩罚错误 token 及其后缀。但现有结果并不支持它优于普通 totalnorm：

- 相比 `totalnorm`，`errtok` 在 `NDCG@10` 上低 `0.0110`
- 在 `HR@50` 上低 `0.0296`

这说明“错误 token 专属惩罚”这个想法本身未必错，但当前实现、权重比例或归一化方式还没有对齐任务主矛盾。

4.5 reward 线的可信结论

Reward 线现在可以站得住的结论

- *当前最强的无 hint top-10 baseline 仍然是* ***rule-only rerun****。*
- *纯 prefix reward 不能替代 exact reward，只能视作辅助信号候选。*
- *token-level prefix reward 想要可用，关键不在“更细粒度”本身，而在* ***normalization 与 credit assignment 是否稳定****。*

5 主结果二：Hint Scaffold Ablation

Variant	Best ckpt	NDCG@10	HR@10	NDCG@50	HR@50
Dynamic hint + ranking	checkpoint-1665	0.0911	0.1138	0.1058	0.1822
Dynamic hint rule-only (old)	checkpoint-1665	0.0919	0.1160	0.1080	0.1905
Dynamic hint rule-only + gather fix	checkpoint-2997	0.0936	0.1169	0.1083	0.1855
Dynamic hint sid-only	checkpoint-2394	0.0921	0.1155	0.1065	0.1830
Fixed hint mixed-single	checkpoint-3326	0.0953	0.1193	0.1114	0.1938
Fixed hint task+index early	checkpoint-666	0.0894	0.1131	0.1051	0.1864
Fixed hint taskfix b16	checkpoint-2997	0.0931	0.1189	0.1094	0.1941

表 2: Hint 主结果表。来源：S1，按 best `NDCG@10` 选点。

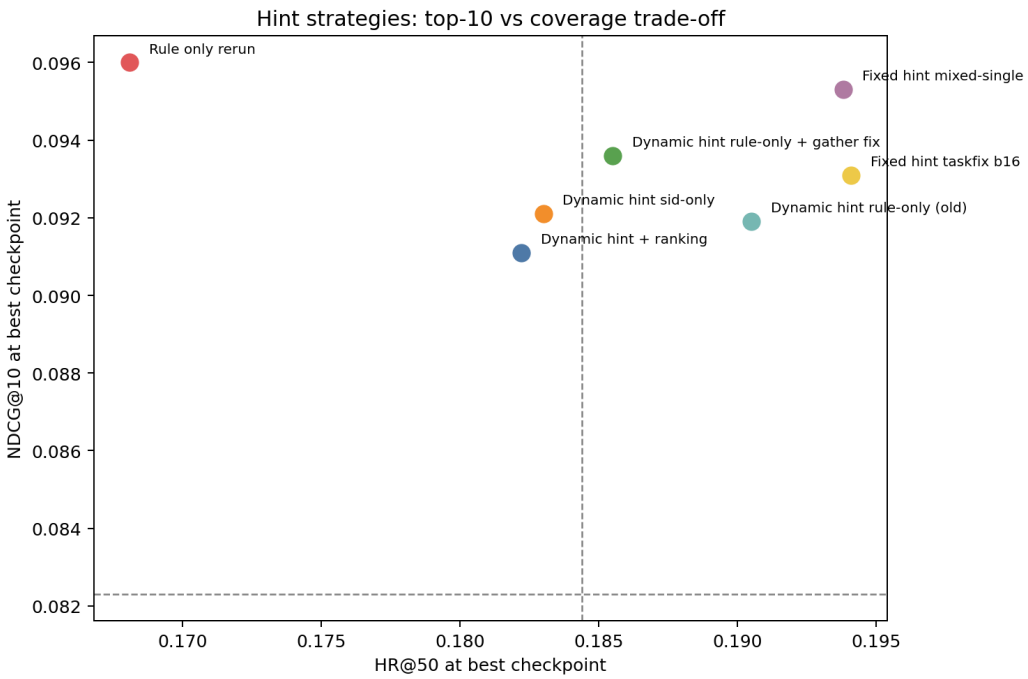


图 2: Hint 方案的 top-10 / coverage trade-off。

[0]来源：作者基于 `experiment_best_metrics.csv` 自绘。

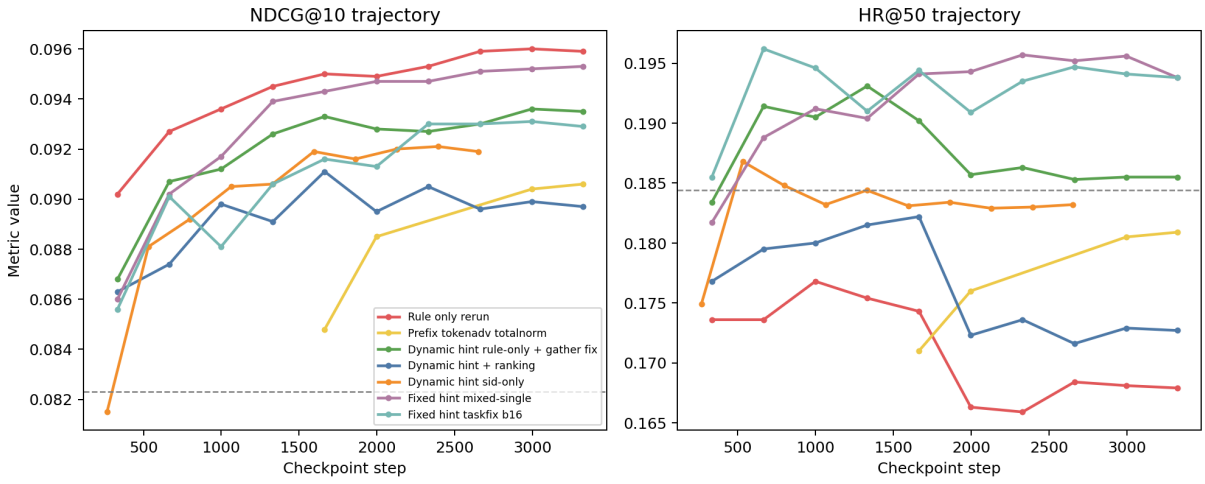


图 3: 代表性实验的训练轨迹。左图为 NDCG@10，右图为 HR@50。

5.1 fixed hint 是当前最平衡、也最值得写成主结果的方向

如果只看 best NDCG@10，plain rule-only 仍然略高于 fixed hint mixed-single：

- `rule-only rerun: NDCG@10=0.0960`
- `fixed hint mixed-single: NDCG@10=0.0953`

但这个差距只有 0.0007，几乎可以理解成“固定 scaffold 没有牺牲 top-10”。与此同时，fixed hint mixed-single 带来了明显更强的 coverage：

- `NDCG@50 +0.0044` relative to plain rule-only
- `HR@50 +0.0257` relative to plain rule-only

这条结果很关键，因为它说明 fixed hint 不是“靠 hint 把 top-10 做差了但 coverage 变好”，而是“**几乎保住 top-10，同时显著修复 coverage**”。

5.2 task-aware fixed hint map 更安全，且仍然保住了 coverage

`fixed hint taskfix b16` 是更值得信的工程版本，因为它对齐了 `task+index` 复合 sample key，而不是单独使用 `index`。这条线的 best 点是：

- `NDCG@10=0.0931`
- `HR@50=0.1941`

和旧的 fixed hint mixed-single 相比：

- `NDCG@10` 低 `0.0022`
- `HR@50` 高 `0.0003`

这里真正被修掉的不是“泄漏”，而是 **cross-task index collision**。相同 `index` 在不同 task 上可能对应不同最小有效 hint depth；旧版 index-only map 会把它们错误地折叠成一个共享 depth。

[0]来源：作者基于 `metrics.json` 时间序列自绘。灰色虚线为 SFT 基线。

5.3 旧 fixed 错版为什么反而更好：它本质上是一个更浅的 hint curriculum

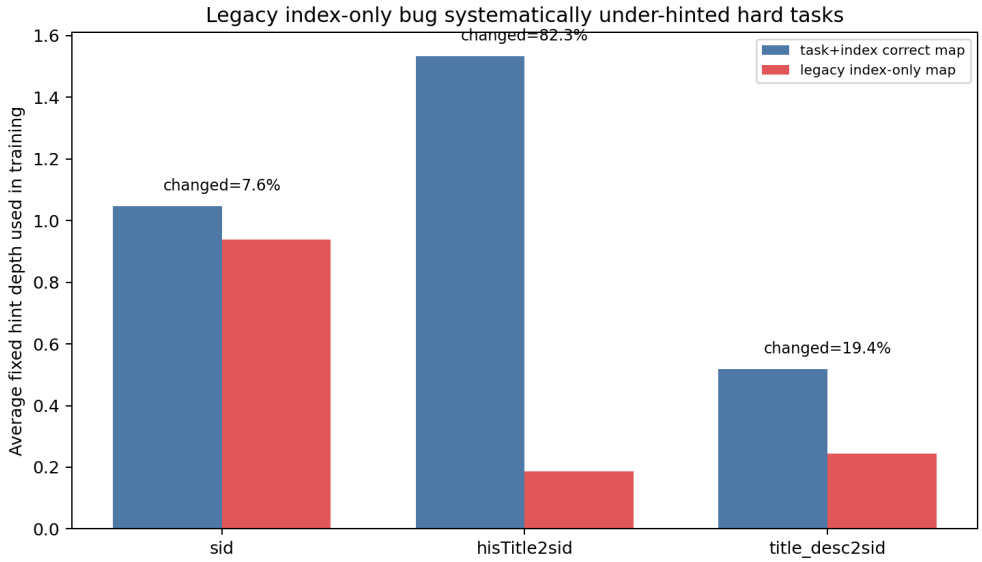


图 4: 旧的 index-only fixed hint bug 会系统性压浅 hard task 的 hint 深度。

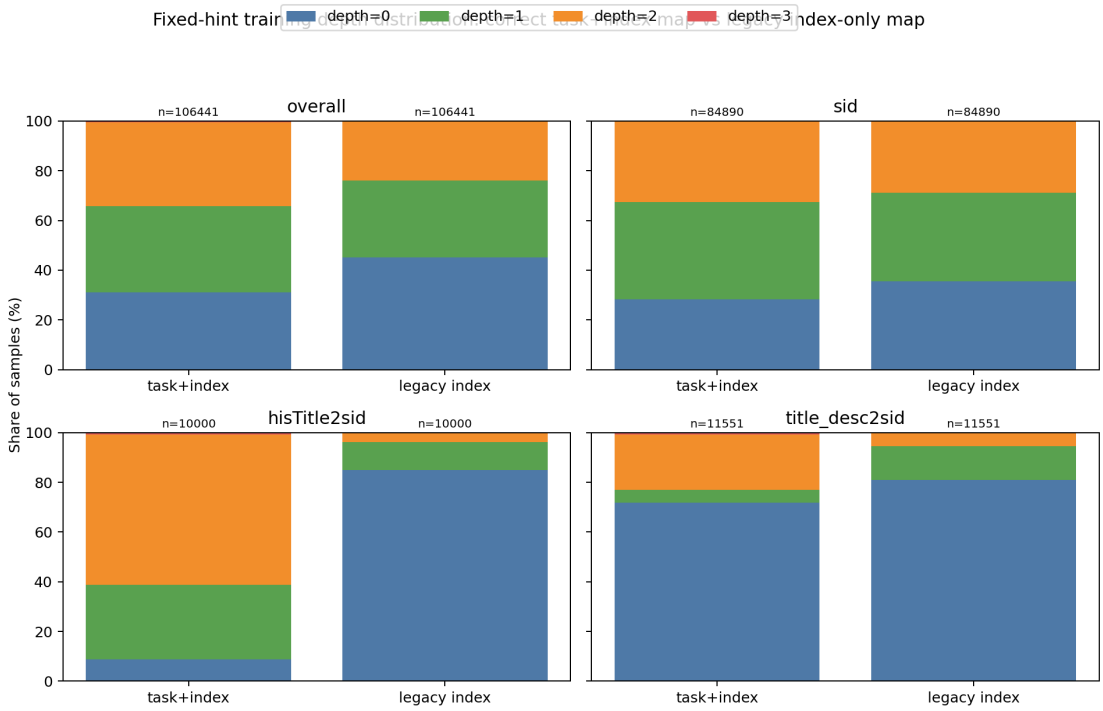


图 5: 旧 fixed bug 与修复后 fixed 在 overall 及各 task 上的 hint-depth 分布。

[0]来源：作者基于 `instruments_grec_beam16_hint_difficulty_table.csv` 重建“legacy index-only map”与“task+index correct map”后自绘。

[0]来源：作者基于 `fixed_hint_bug_depth_distribution.csv` 自绘。

Scope	Version	depth=0	depth=1	depth=2	depth=3
overall	task+index	31.1%	34.5%	33.9%	0.4%
overall	legacy index	45.0%	30.9%	23.8%	0.2%
sid	task+index	28.3%	39.0%	32.4%	0.3%
sid	legacy index	35.4%	35.6%	28.7%	0.3%
hisTitle2sid	task+index	8.7%	30.1%	60.5%	0.8%
hisTitle2sid	legacy index	85.0%	11.2%	3.8%	0.0%
title_desc2sid	task+index	71.8%	5.2%	22.2%	0.8%
title_desc2sid	legacy index	80.9%	13.7%	5.4%	0.0%

表 3: 旧 fixed bug 与修复后 fixed 的训练期 hint-depth 比例。来源：`fixed_hint_bug_depth_distribution.csv`。

把离线难度表重新按“旧 bug 逻辑”折叠一遍后，可以看到这个 bug 不是轻微扰动，而是大幅改变了训练期 scaffold 强度：

- 全体样本里有 `15.94%` 的 hint depth 被压浅。
- 对 `sid`，变化率只有 `7.64%`，平均深度从 `1.048` 降到 `0.938`。
- 对 `title_desc2sid`，变化率是 `19.41%`，平均深度从 `0.519` 降到 `0.245`。
- 对 `hisTitle2sid`，变化率高达 `82.35%`，平均深度从 `1.533` 直接降到 `0.188`。

也就是说，旧 fixed 不是“hint 更准”，恰恰相反，它对 hardest task 大规模**少给了 hint**。仅 `hisTitle2sid` 一项里，就有：

- `2481` 个样本从 `1->0`
- `5090` 个样本从 `2->0`
- `586` 个样本从 `2->1`

如果直接看 hint 比例，这个 bug 的形状更明显：

- overall 上，旧版把 `depth=0` 比例从 `31.1%` 抬到 `45.0%`，同时把 `depth=2` 比例从 `33.9%` 压到 `23.8%`。
- 对 `sid`，变化不算极端，但也把 `depth=0` 从 `28.3%` 抬到了 `35.4%`。
- 对 `title_desc2sid`，旧版把大量样本从 `depth=2` 挤到 `depth=1` 或 `depth=0`，`depth=2` 比例从 `22.2%` 掉到 `5.4%`。
- 对 `hisTitle2sid`，旧版几乎把它整个“洗平”了：`depth=0` 直接从 `8.7%` 暴涨到 `85.0%`，`depth=2` 从 `60.5%` 跌到 `3.8%`。

这就给了“错版 fixed 反而更好”一个更可信的解释：旧 bug 意外形成了一个**更浅、更硬的 scaffold curriculum**。它让 hardest task 在训练时更接近 no-hint 条件，因此虽然优化更难，却可能更符合最终无 hint 评测的 transfer 目标。

这一点也能从逐 checkpoint 差值看出来：修复后的 `taskfix` 版本在很早期的 `HR@50` 往往不差，甚至在 `333/666/999` 这些点上略高；但从 `999` 之后开始，旧 fixed 在 `NDCG@10` 上几乎持续领先 `0.0017--0.0034`。更像是：

- 更深、更准确的 hint 让训练更容易；
- 但也让训练目标离最终 no-hint eval 更远。

5.4 dynamic hint 有价值，但 canonical 版本应以 gather-fix 为准

dynamic hint line 的可信结论分三层：

- **第一层**: 旧 dynamic 结果已经被 3 月 20 日的 gather fix supersede，不应再作为 canonical baseline。
- 它**确实能把 coverage 拉回到高于 SFT 的区间**，plain rule-only 做不到这一点。
- 但它在当前结果里**仍然没有超过 fixed hint**。

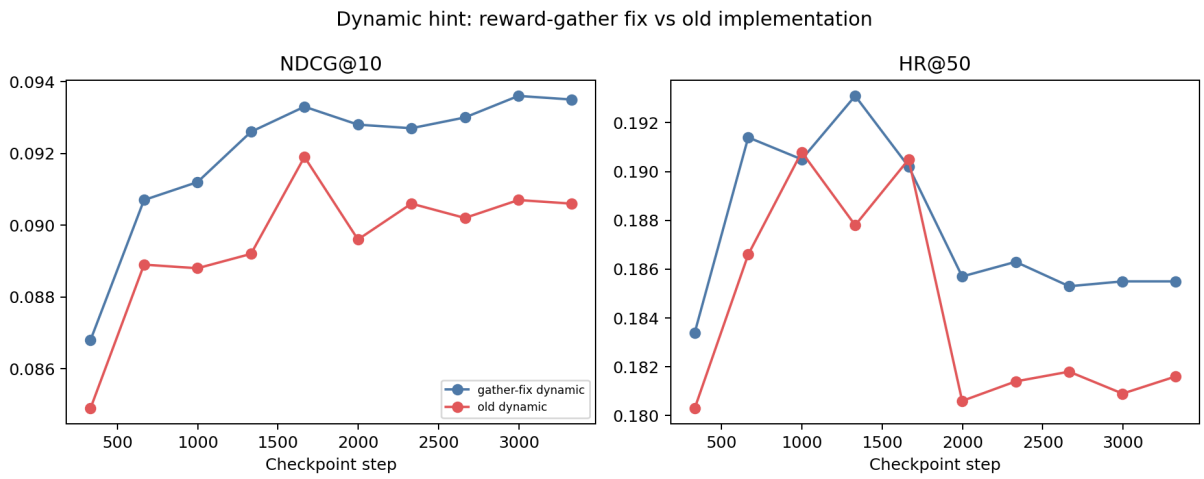


图 6: dynamic hint 的 gather-fix 版本在全部对齐 checkpoint 上都优于旧版本。

旧 dynamic 与 gather-fix dynamic 的逐 checkpoint 对比非常干净：在全部 `333--3326` 对齐 checkpoint 上，`gather-fix` 版本的 `NDCG@10` 都更高，提升区间是 `+0.0014` 到 `+0.0034`；`HR@10` 也全部更高，提升区间是 `+0.0006` 到 `+0.0048`。这说明 dynamic 旧版确实低估了 dynamic 路线本身的上限。

最好的 dynamic hint rule-only + gather fix 达到：

- `NDCG@10=0.0936`
- `HR@50=0.1855`

相比 fixed hint mixed-single：

- `NDCG@10` 低 `0.0017`
- `HR@50` 低 `0.0083`

[0]来源：作者基于 `metrics_inventory.csv` 中两条 dynamic run 的对齐 checkpoint 自绘。

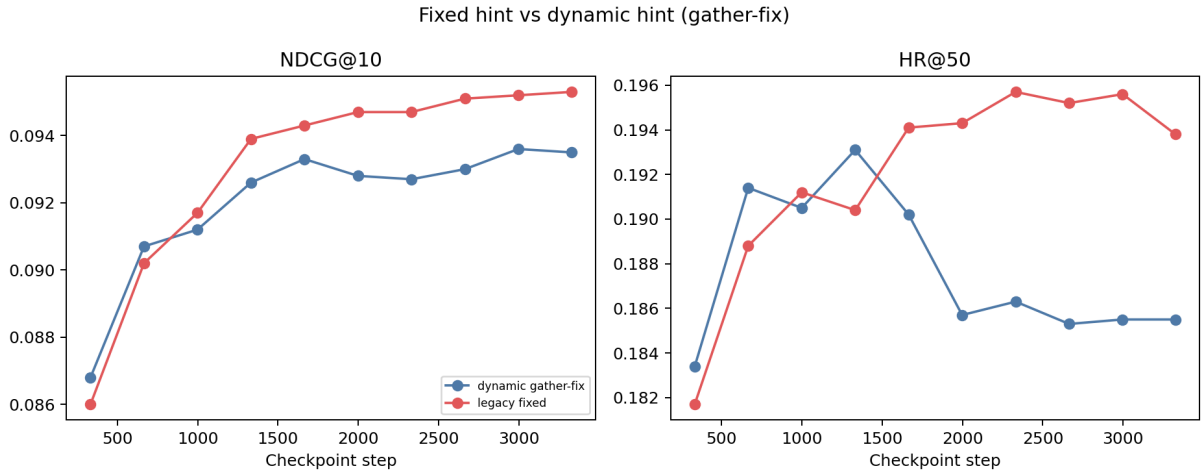


图 7: fixed hint 与 dynamic hint(gather-fix) 的逐 checkpoint 对比。

更细地看时间轨迹，会发现 dynamic 并不是从第一步就输 fixed。恰恰相反：

- 在 `checkpoint-333` 和 `666`，dynamic gather-fix 在 `NDCG@10` 上分别高 `0.0008` 和 `0.0005`。
- 从 `checkpoint-999` 开始，fixed 反超并保持到结束。
- 到后半程 `1998--3326`，fixed 对 dynamic 的优势大致稳定在 `NDCG@10 +0.0016--0.0021`、`HR@50 +0.0083--0.0101`。

这组形状非常像下面这个机制：

- dynamic 的 on-policy hint selection 在早期确实更贴合当前策略，所以起步略快；
- 但 dynamic 的 stage gate 是 `rule_hit_any`。在当前 item-trie 下，完整正确 item 本质上只有一个，所以这个 gate 几乎等价于“那唯一正确 beam 这一步有没有刚好出现”；
- 一旦这个 one-hit 事件在浅 stage 发生，dynamic 就直接锁定该 depth，并丢弃更深 stage 的候选；因此它优化的是“**最早撞中 1 条 beam 的深度**”，而不一定是“**最适合训练的深度**”；
- hint depth 在线变化本身不是问题，问题是它现在对这个 one-hit 事件过于敏感：同一样本可能因为这一轮 16-beam 里是否冒出那一条 exact beam，而在 0/1/2/3 之间跳动；
- fixed 虽然 map 可能陈旧，但在当前 `2 epoch + 1e-5` 的训练幅度下，stale-map drift 似乎没有这种 one-hit gate 带来的 depth 抖动更伤。

所以更准确的表述不是“dynamic 理论错了”，而是“**在当前训练强度下，策略漂移还不足以压过 one-hit gate 引入的 depth 抖动**”。这也解释了为什么 fixed 最终更强。

5.5 sid-only 更直接对齐评测任务，但目前仍未优于 mixed-task

这里需要修正我上一版报告里的一个判断。数据生成脚本 `preprocess_data_sft_rl.py` 明确表明：

- 默认 mixed RL 训练集包含 `task1_sid_sft`、`task4_hisTitle2sid`、`task5_title_desc2sid`。

[0]来源：作者基于 `metrics_inventory.csv` 对齐 checkpoint 后自绘。

- 但 `rl_valid`、`rl_test` 只保留 `task1_sid_sft`。
- `scripts/run_instruments_preprocess_grec_rl_sid_only.sh` 则通过 `RL_ONLY_TASK1=true` 只保留 task1 进入 RL 训练。

因此，你说得对：`sid-only` 训练确实**更直接对齐了最终评测任务**，不是我上一版写的那种“因为 benchmark 是 mixed-task 所以 sid-only 吃亏”。

即便如此，`dynamic hint sid-only` 的 best 点仍然弱于 mixed-task 的 `dynamic hint rule-only + gather fix`：

- `NDCG@10` 低 `0.0015`
- `HR@10` 低 `0.0014`
- `NDCG@50` 低 `0.0018`
- `HR@50` 低 `0.0025`

这件事更合理的解释不是“sid-only 对错了 benchmark”，而是：

- task4 / task5 虽然不是最终评测任务，
- 但它们和 task1 共享同一套 SID 目标空间，
- 很可能在训练期提供了额外的 prefix routing / SID token semantics 监督。

因此，当前证据更像是在支持“**辅助任务带来了正迁移**”，而不是支持“mixed-task 只是因为口径更贴近评测”。不过这条仍然不是完美 apples-to-apples，因为 sid-only run 的 checkpoint 节奏与样本量都变了，所以最稳妥的后续工作仍然是做统一训练预算下的 paired rerun。

5.6 ranking 没有显示出明确收益，而且有机制层面的解释

当前最接近的实证对比是 dynamic hint 组：

- `dynamic hint + ranking`: `NDCG@10=0.0911`, `HR@50=0.1822`
- `dynamic hint rule-only + gather fix`: `NDCG@10=0.0936`, `HR@50=0.1855`

排名 reward 版本在四个主指标上都更差。这里需要修正我上一版报告里的保守说法：**这条证据比“提示性证据”更强**。原因是 `ranking dynamic hint` 的 launcher 是 2026-03-28 才新增的，而 dynamic gather fix 本身在 2026-03-20 已经进入 trainer。也就是说，ranking dynamic 不是跑在“老 bug 版 dynamic”上的，它已经天然继承了 gather-fix 之后的 dynamic trainer。

即便如此，代码层也支持“ranking 不太对主矛盾”这个判断：当前 ranking reward 只在 group 里已经存在 exact hit 时，才会给 non-exact beams 一个 rank-based negative signal；对于 hardest case 的“整组都没有 exact hit”，ranking reward 直接归零。也就是说，它主要在“已经有成功样本的组里做 sharpening”，而不是在“完全没学会的组里提供更有用的梯度”。

6 为什么 fixed 当前比 dynamic 更好

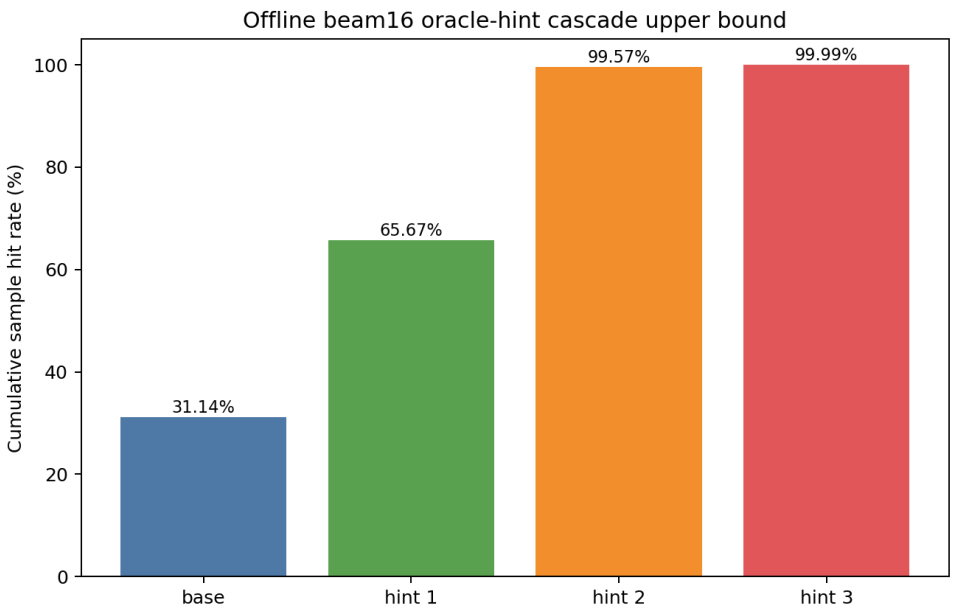


图 8: 离线 beam16 oracle-hint cascade 的 cumulative sample hit upper bound。

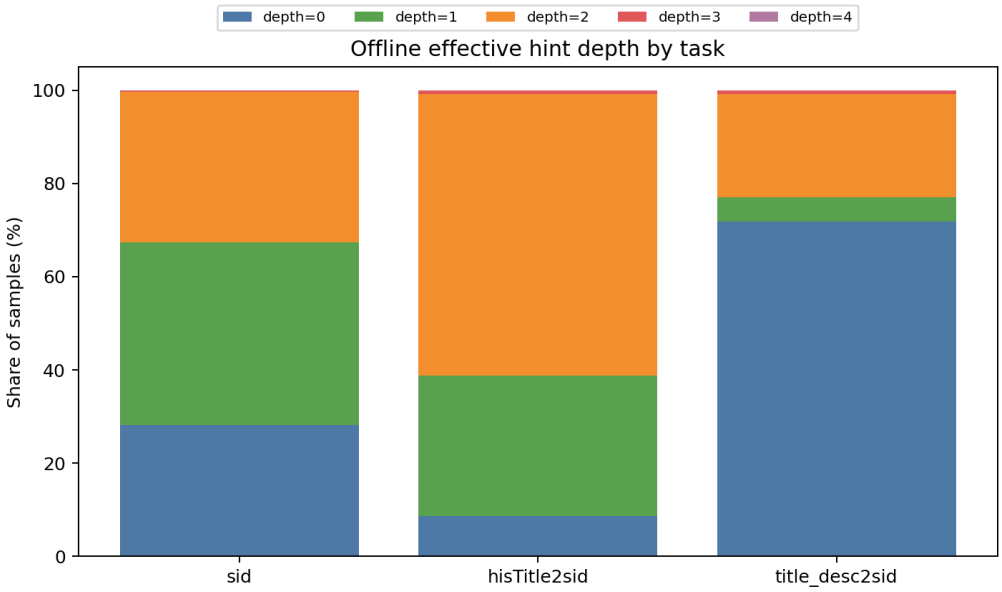


图 9: 离线 effective hint depth 的 task-level 分布。

6.1 先把误解去掉：dynamic 的“变化”不是错，问题在于 gate 与优化目标不一致

你指出得对，dynamic hint 的 hint depth 随当前策略变化，本来就是这个方法的定义，不应该被我上一版写成“变化本身有问题”。真正的问题不在“会变化”，而在于**现在 dynamic 用来决定 hint depth 的 gate，与后续真正被优化的 GRPO 信号不是同一个东西**：

[0]来源：`temp/rl_beam_hint/instruments_grec_beam_hint_cascade_20260314_task_index_fix_export_summary.json`。

[0]来源：`output/jupyter-notebook/genrec-hint-cascade-artifacts/instruments_grec_beam16_hint_difficulty_table.csv`。

- 代码里 dynamic stage 的停止条件是 `group_rule_hit_any`；
- 对每个 prompt group，只要当前 stage 的 16 条 beam 里有 **任意一条** exact hit，就直接选中这一 stage，后续更深的 stage 不再看；
- 这意味着 dynamic 实际上选的是 **minimum-any-hit depth**。

但 pure `rule_only` 下，GRPO 真正优化的是所选 stage 的**整组 reward / advantage 分布**。如果某个 stage 上 16 条 beam 里有 k 条 exact hit，那么：

$$r_i \in \{0, 1\}, \qquad \bar{r} = \frac{k}{16}, \qquad A_i = r_i - \bar{r}$$

- r_i：第 i 条 beam 的 rule reward
- $\bar{r}$：这一整个 prompt group 的平均 reward
- A_i：进入 GRPO 的组内 centered advantage
- k：这一 stage 上 exact-hit beam 的数量

所以 dynamic 当前更关键的问题不是“hint depth 会动”，而是：

- gate 只看“$k > 0$ 是否第一次成立”；
- 但训练真正关心的是所选 stage 的整个 reward distribution，也就是 k 到底有多大；
- 如果浅 stage 只有 $k = 1$，而更深 stage 会给 $k = 4$、$k = 8$ 甚至更多 exact-hit beams，dynamic 也完全看不到。

如果还沿用“variance”这个词，这里更准确指的是一种**非平滑的 prompt-schedule 跳变**：不是 beam search 本身随机，而是同一样本一旦在浅 stage 第一次让 k 从 0 变成正数，dynamic 就会切到更浅的 conditioning。

6.2 fixed 比 dynamic 强，不是因为 fixed 理论更对，而是因为当前 dynamic gate 太脆

把 fixed 与 dynamic 放在一起看，更合理的解释是：

- fixed 提供的是一个**稳定但可能偏差**的 scaffold；
- dynamic 提供的是一个**更自适应但 conditioning schedule 更不平滑**的 scaffold；
- 在当前 `2 epoch + 1e-5` 的训练幅度下，稳定性收益似乎大于自适应收益。

这也解释了为什么 dynamic 在早期并不差，甚至略好，而后期被 fixed 反超：

- 早期模型很弱时，on-policy 动态 hint 确实更贴合当前能力，所以起步更快；
- 但随着训练推进，dynamic 会越来越早地在浅 stage 触发 $k > 0$，从而提前切到更浅的 prompt conditioning；
- 这使 dynamic 更像是在优化“当前策略最小需要几层 hint 才能至少打中 1 条 beam”，而不是优化“哪个 hint depth 最利于形成稳定、可迁移的训练分布”。

换句话说，dynamic 现在选的是 **minimum hit depth**，而不是真正意义上的 **best training depth**。这两者并不等价。

6.3 为什么错版 fixed 反而更好：少一些 hint 可能更接近最终目标，但前提是要稳定

旧的 index-only bug 给 hardest task 大幅少给了 hint，但它仍然是**稳定地少给**，不是像 dynamic 那样随 step 抖动。这个现象更像是在无意中做了一种“浅 scaffold curriculum”：

- 比正确 task+index map 更少的 hint，使训练分布更接近最终无 hint 评测；
- 但因为这个浅 hint 是离线固定的，所以不会像 dynamic 那样受 one-hit 门控影响来回跳。

因此，当前最合理的理解不是“dynamic 本该更好却失败了”，而是：

- **hint 少一点**可能对 no-hint transfer 有好处；
- **hint 深度稳定**也有明显好处；
- 旧 fixed 恰好踩中了“更浅但更稳”这个组合；
- 修复后的 fixed 变得“更准但更深”，dynamic 则是“可能更浅但不够稳”。

这三者的 trade-off，刚好能解释当前排序：旧 fixed 最强，taskfix fixed 次之，dynamic gather-fix 再后。

6.4 如果要让 dynamic 真正赢 fixed，它需要改 stopping signal，而不是只靠在线变化

基于现在的代码，我认为 dynamic 要真正超过 fixed，至少要改掉现在这个 `rule_hit_any` gate。更合理的候选包括：

- 不再用“是否出现 1 条 exact hit”作为 stopping 条件，而用 `rule_hit_count`、`group mean reward`、`mean prefix progress` 或 top-k beam 质量。
- 把 dynamic depth 和一个离线 teacher map 混合，而不是完全由当步 rollout 决定。
- 对同一样本引入 depth hysteresis/EMA，避免 0/1/2/3 之间来回抖动。

也就是说，dynamic 的问题不是“它会变”，而是“**它现在用错了 gate**”。

7 代码层解释：为什么 ranking 不明显、为什么 prefix shaping 容易不稳定

7.1 ranking reward 对 hardest case 几乎没有帮助

`rewards/ranking_reward.py` 的 reward setup 本质上是：

```python
if mode == "rule_only":
    return [rule_reward], None
if mode == "prefix_only":
    return _append_rule_reward_probe([prefix_func, ndcg_func], None)
if mode == "ranking":
    return [rule_reward, get_ndcg_rule_reward(num_beams)], None
```

Listing 1: 当前 reward mode 分支的核心定义

而 `ndcg_rule_reward` 的行为不是“prefix 越长越奖励”，而是“**当组内已有 exact hit 时，对 non-exact beams 按 rank 给负信号**”。更关键的是，GRPO 最终优化的不是原始 reward，而是组内 advantage：

```
rewards = (rewards_per_func * self.reward_weights.to(device).unsqueeze(0)).nansum(dim=1)
mean_grouped_rewards = rewards.view(-1, self.num_generations).mean(dim=1)
mean_grouped_rewards = mean_grouped_rewards.repeat_interleave(self.num_generations, dim=0)
advantages = rewards - mean_grouped_rewards
```

Listing 2: 当前 dynamic/fixed trainer 与普通 GRPO 一样，最终用组内 centered advantage

所以 ranking 在这里的作用实际上更弱：

- 对完全没解开的 group，ranking reward 直接没有额外帮助；
- 对已经有 exact hit 的 group，它主要只是**重新排列这一组 rollout 之间的相对次序**；
- 它并不增加正样本数量，也不解决 zero-hit group 没梯度的问题；
- 在 current setup 下，它本质上更像“rollout 内部排序信号”，而不是能改变样本级可学性的主信号。

这与离线 bundle 的难度画像正好冲突：当前 hardest case 主要集中在前两位 SID token 的 routing 问题，而不是“已经有 exact hit 后，怎样把 beam rank 排得更尖”。

7.2 prefix shaping 的问题不在“粒度”，而在“credit assignment”

`token_prefix_grpo_trainer.py` 里，matched prefix token 默认得到的是 `1/L` 级别的局部正信号；如果采用 `token_adv_total_token_normalize`，则会把 prefix token reward 与 NDCG token reward 先合成，再按 token 位置做 group normalization。代码本身已经说明了这个设计非常敏感：

- raw token-level 组合会把不同 token 位置、不同 rollout 的信号直接混在一起；
- totalnorm 至少让这些信号在同一 token-wise normalization 语义下比较；
- errtok 再进一步改变了负信号的覆盖范围，但当前实验没有显示出收益。

所以这条线当前最好的表述不是“我们已经找到最优 token reward”，而是“**我们证明了 token-level prefix shaping 能否成立，主要取决于 normalization 细节；没有正确 normalization，会直接崩**”。

8 我们现在最可信的 contribution 是什么

可以写进论文的三点 contribution

- ***Contribution 1: task-aware offline hint analysis* 识别了 *GenRec semantic-ID RL* 的主瓶颈**。离线 *beam16 cascade* 显示，大多数样本的主要困难集中在前两位 *SID token*；不同 *task* 的难度形状明显不同，不能被单一局部树结构完全解释。

- ***Contribution 2: oracle scaffold* 比复杂 *reward shaping* 更有效**。现有实验表明，*fixed hint* 能以几乎不损失 *top-10* 的代价，显著修复 *plain rule-only* 的 *coverage* 损失；这比当前任何单纯改 *reward* 形状的方案都更稳定、更有故事性。
- ***Contribution 3:* 我们把"稀疏 *reward* 的问题"重新定位成"可迁移 *scaffold* 学习的问题"**。离线上界几乎 *100%* 可解，但 *no-hint* 在线评测仍有明显 *gap*；这说明真正要解决的不是简单地换 *dense reward*，而是如何把 *offline scaffold upper bound* 转移成 *test-time* 无 *hint* 能力。

如果现在就要为论文定主线，我会建议主叙事是：

- 先用离线 hint bundle 证明 **semantic-ID recommendation 的 hardest routing 集中在 prefix**；
- 再用 RL ablation 证明 **复杂 reward shaping 并不稳定，oracle scaffold 更有效**；
- 最后把 fixed hint 看成第一版可行系统，把 future work 指向"如何把 oracle scaffold 逐步蒸馏成无 hint policy"。

9 后续应该怎么做

9.1 优先级最高：把 fixed hint 变成真正的主线方法

最值得继续加资源的不是再开更多 prefix reward 变体，而是 fixed hint 主线，具体建议：

- 以 `taskfix-b16` 作为更可信的工程基线继续长训和复现。
- 单独做 `fixed hint apply to eval = true/false` 的 paired evaluation，用于拆出"hinted test-time upper bound"和"no-hint transfer gain"。
- 增加一条 **hint dropout / hint annealing** 线，验证能否把训练期 scaffold 更好地蒸馏回无 hint 测试表现。

9.2 第二优先级：把 dynamic hint 变成 scheduler ablation，而不是主结果

dynamic hint 目前最合理的定位不是主方法，而是：

- 证明 online scheduler 方向有价值；
- 提供对 fixed hint map 的 online 近似；
- 为后续 adaptive scaffold policy 提供原型。

但在它真正成为主结果之前，至少要补两件事：

- 把 dynamic 的 stopping criterion 从 `rule_hit_any` 升级成更稳的质量统计，再看它是否还能输给 fixed；
- 用与 `rule-only dynamic hint` 完全对齐的 seed / 训练预算 / 保存节奏，再做一次 `ranking dynamic hint` 配对复现，确认当前劣势不是偶然种子；
- 统一 mixed-task 与 sid-only 的训练步数、checkpoint 采样节奏，再做更干净的 task ablation。

9.3 第三优先级：reward 线只保留最必要的四条

当前 reward 线已经足以收敛，不需要继续大规模铺开：

- `rule-only rerun`
- baseline mixed GRPO
- `prefix tokenadv totalnorm`
- fixed hint taskfix

这四条已经能覆盖：

- 稀疏 exact reward baseline
- mixed reward baseline
- “reward shaping 能否救回来”的最好版本
- “scaffold 是否比 shaping 更有效”的最强证据

9.4 任务设计层面：不要过早只押 sid-only，要先做 clean ablation

从离线 bundle 和 sid-only run 的结果一起看，更合理的方向是：

- 不要先验地断定 sid-only 一定更好或 mixed-task 一定更好；
- 在统一训练预算下，把 sid-only 与 mixed-task 做真正配对实验；
- 明确引入 task-aware scaffold depth、task-aware sampling 或 task-aware curriculum；
- 如果要做更细的 RL objective，先按 task 观察哪一类样本主要失败在第几层 SID token，再决定是否需要不同 reward/hint policy。

10 结论与后续问题

10.1 已经被强支持的结论

- fixed hint 是当前最值得继续推进的主线方法，它几乎保住了 best top-10，同时显著修复了 rule-only 的 coverage 损失。
- 复杂 reward shaping 目前没有展示出超越 scaffold 的稳定收益；其中 totalnorm 是 token-level shaping 可用的必要条件之一。
- dynamic 当前输给 fixed，更像是输在 `rule_hit_any` 这种 one-hit gate 带来的 depth 抖动，而不是输在“在线自适应”这个想法本身。
- 虽然评测任务是 task1-only，但 mixed-task 训练目前仍略优于 sid-only，说明 task4/task5 可能带来了正的辅助迁移。

10.2 较大概率为真，但还需要补 cleaner ablation 的结论

- ranking reward 在当前任务里很可能不是主矛盾，至少现有结果没有显示收益。
- task-aware fixed hint map 的小幅 top-10 回落更像“修复 index collision 后，hint 变深带来的 transfer 代价”，而不是方法本身无效。
- dynamic hint 的最终潜力还没被跑干净，但它更像 scheduler/teacher，而不是当前最强成品方法。

10.3 仍然未知、值得马上验证的问题

- fixed hint 的收益里，有多少来自 train-time scaffold 本身，有多少可以转化成 no-hint inference gain?
- hint dropout、hint annealing 或 teacher-student distillation 能否把 fixed hint 的 coverage 优势更多转化成无 hint 的 top-10 改善?
- ranking reward 若结合 prefix progress，而不是只在 exact-hit group 内做 rank penalty，是否会更贴近真实瓶颈?

A 用户指定目录的分类索引

Label	Classification	Best ckpt	Best NDCG@10	Best HR@50
Dynamic hint + ranking	main_hint	checkpoint-1665	0.0911	0.1822
Dynamic hint rule-only (old)	main_hint_superseded_by_gather_fix	checkpoint-1665	0.0919	0.1905
Dynamic hint rule-only + gather fix	main_hint	checkpoint-2997	0.0936	0.1855
Dynamic hint sid-only	main_hint	checkpoint-2394	0.0921	0.1830
Fixed hint mixed-single	main_hint	checkpoint-3326	0.0953	0.1938
Fixed hint task+index early	early_taskfix_run	checkpoint-666	0.0894	0.1864
Fixed hint taskfix b16	main_hint	checkpoint-2997	0.0931	0.1941
Prefix old alias	legacy_alias	checkpoint-1665	0.0863	0.1631
Prefix backup	partial_backup	checkpoint-333	0.0815	0.1608
Prefix seq only broken run	superseded_broken_run	checkpoint-1332	0.0389	0.1023

Label	Classification	Best ckpt	Best NDCG@10	Best HR@50
Rule only old run	superseded_old_run	checkpoint-333	0.0761	0.1494
Baseline mixed GRPO	main_reward	checkpoint-1665	0.0952	0.1696
Rule only rerun	main_reward	checkpoint-2997	0.0960	0.1681
Prefix only	main_reward	checkpoint-1332	0.0862	0.1612
Prefix seq only rerun	main_reward	checkpoint-1332	0.0857	0.1636
Prefix token only	main_reward	checkpoint-1665	0.0815	0.1668
Prefix tokenadv raw	main_reward	checkpoint-999	0.0476	0.0991
Prefix tokenadv total-norm	main_reward	checkpoint-3326	0.0906	0.1809
Prefix tokenadv errtok	main_reward	checkpoint-999	0.0796	0.1513
Prefix backup single checkpoint	single_checkpoint_backup	checkpoint-333	0.0815	0.1608

B Legacy / Superseded Run 结果表

Variant	Best ckpt	NDCG@10	HR@10	NDCG@50	HR@50
Prefix old alias	checkpoint-1665	0.0863	0.1060	0.0985	0.1631
Prefix backup	checkpoint-333	0.0815	0.0998	0.0946	0.1608
Prefix seq only broken run	checkpoint-1332	0.0389	0.0494	0.0504	0.1023
Rule only old run	checkpoint-333	0.0761	0.0907	0.0885	0.1494

表 5: legacy / superseded run 只用于解释历史演化和 bug 修复，不作为主结论。

C 来源清单

ID	模态	标题	路径
S1	data	Checkpoint metrics inventory	/Users/fanghaotian/Desktop/src/GenRec/docs/deepresearch/genrec_rl_study_2026-03-28/data/m
S2	data	Beam16 hint difficulty table	/Users/fanghaotian/Desktop/src/GenRec/output/jupyter-notebook/genrec-hint-cascade-artifac

ID	模态	标题	路径
S3	data	Beam16 cascade summary	/Users/fanghaotian/Desktop/src/GenRec/temp/rl_beam_hint/instruments_grec_beam_hint_cascad
S4	data	Task-aware fixed hint map	/Users/fanghaotian/Desktop/src/GenRec/output/jupyter-notebook/genrec-hint-cascade-artifac
S5	code	Reward setup implementation	/Users/fanghaotian/Desktop/src/GenRec/rewards/ranking_reward.py
S6	code	Token-level prefix trainer	/Users/fanghaotian/Desktop/src/GenRec/token_prefix_grpo_trainer.py
S7	code	Main RL trainer entrypoint	/Users/fanghaotian/Desktop/src/GenRec/trl_trainer.py
S8	code	Dynamic/fixed hint trainer internals	/Users/fanghaotian/Desktop/src/GenRec/fixed_hint_grpo_trainer.py
S9	code	External evaluation pipeline	/Users/fanghaotian/Desktop/src/GenRec/evaluate.py
S10	doc	Main results weekly report	/Users/fanghaotian/Desktop/src/GenRec/docs/genrec_main_results_weekly_2026-03-18.zh.md
S11	doc	Reward form analysis	/Users/fanghaotian/Desktop/src/GenRec/docs/genrec_rl_reward_forms_2026-03-13.md
S12	doc	Dynamic vs fixed hint implementation note	/Users/fanghaotian/Desktop/src/GenRec/docs/genrec_dynamic_fixed_hint_metrics_2026-03-17.z